

NHCX Code Snippet References For Payload Preparation

1. Prepare FHIR resource

Refer the NRCES technical specifications for the respective use case . For any query regarding this , please connect with NRCES team at mail id- nrc-help@cdac.in.

The entire payload comprises the below parts separated by “.”:

- Protected Header
- FHIR Bundle encrypted using the public key

2. Preparing protected headers

```
{"alg":"RSA-OAEP-256","enc":"A256GCM","x-hcx-api_call_id":"bbea0ea1-2750-4ec8-8974-d3edc7030913","x-hcx-workflow_id":"1","x-hcx-request_id":"1113f541-6387-4c2a-ad3e-4bebc5a9fe36","x-hcx-status":"request.initiate","x-hcx-timestamp":"2023-08-02T14:57:40+0530","x-hcx-sender_code":"1000000046@sbx","x-hcx-recipient_code":"1000000109@sbx","x-hcx-correlation_id":"3a2ba13b-04f5-4943-9b50-a1334e55c90e"}
```

Note -This protected header needs to be encoded in base64 format before putting into the payload part.



3. Encryption of Payload (public key.zip)

Headers: Protected Header shared in point 2

payloadMain: FHIR bundle that is prepared

certificatepath: Path where the Public certificate of the receiver is stored in the system.

```
private String encryptPayload(Map<String, Object> headers, Map<String, Object> payloadMain, String certificatepath) {
```

```
logger.info("-- Encrypting payload --");
```

```
JWEPayloadUtil jweRequest = new JWEPayloadUtil();
```

```
String encryptedMsg = null;
```

```
try {
```

```
certificatepath = publicKeyFile;
```

```
encryptedMsg = jweRequest.encryptRequestNew(
```

```

KeyLoader.loadPublicKeyFromX509Certificate(new File(certificatepath)),
headers, payloadMain);

logger.info(String.format("-- Encrypted Payload --> %s", encryptedMsg));

} catch (CertificateException | JOSEException | IOException e) {
    e.printStackTrace();
    logger.info("-- Error while encrypting payload --");
    return null;
}

logger.info(String.format("-- Payload encrypted Successfully --"));
return encryptedMsg;
}

    public static final JWEAlgorithm KEY_MANAGEMENT_ALGORITHM =
JWEAlgorithm.RSA_OAEP_256;
    public static final EncryptionMethod CONTENT_ENCRYPTION_ALGORITHM =
EncryptionMethod.A256GCM;
    public static final EncryptionMethod CONTENT_DECRYPTION_ALGORITHM =
EncryptionMethod.A256GCM;

public String encryptRequestNew(RSAPublicKey rsaPublicKey, Map<String, Object>
headers, Map<String, Object> payload)
    throws JOSEException {

    Map<String, String> encryptedObject = null;
    JWEHeader jweHeader = new JWEHeader.Builder(KEY_MANAGEMENT_ALGORITHM,
CONTENT_ENCRYPTION_ALGORITHM)
        .customParams(headers).build();
    Payload jwePayload = new Payload(payload);

    JWEObjcet jweObject = new JWEObjcet(jweHeader, jwePayload);
    RSAEncrypter rsaEncrypter = new RSAEncrypter(rsaPublicKey);
    jweObject.encrypt(rsaEncrypter);

    String serializedString = jweObject.serialize();

    return serializedString;
}

```

Getting RSA Public Key from Cert File

```

public static RSAPublicKey loadPublicKeyFromX509Certificate(File
publicKeyFile) throws IOException, CertificateException {

```

```

FileReader fileReader = new FileReader(publicKeyFile);

try (PemReader pemReader = new PemReader(fileReader)) {

PemObject pemObject = pemReader.readPemObject();

CertificateFactory certificateFactory =
CertificateFactory.getInstance("X.509");

X509Certificate x509Certificate = (X509Certificate) certificateFactory
.generateCertificate(new
ByteArrayInputStream(pemObject.getContent()));

return (RSAPublicKey) x509Certificate.getPublicKey();
}
}

```

Note- We recommend to store certificate in local location instead of calling /participanthcxs-service/fetch/certs api to get the certificate again and again . Also you can refer sample public and private key available in the attachment

4.Decryption of Payload

```

public Map<String, Object> decryptRequest(RSAPrivateKey rsaPrivateKey, Map<String,
String> encryptedObject) throws ParseException, JOSEException {

    Map<String, Object> headers = null;
    Map<String, Object> payload = null;

    JWEObjcet jweObjcet = new JWEObjcet(new
Base64URL(encryptedObject.get("protected")),
        new Base64URL(encryptedObject.get("encrypted_key")),
        new Base64URL(encryptedObject.get("iv")),
        new Base64URL(encryptedObject.get("ciphertext")),
        new Base64URL(encryptedObject.get("tag")));
    JWEDecrypter jweDecrypter = new RSADecrypter(rsaPrivateKey);
    jweObjcet.decrypt(jweDecrypter);

    headers = jweObjcet.getHeader().toJSONObject();
    payload = new HashMap<>(jweObjcet.getPayload().toJSONObject());

    return payload;
}

```

